AMSS 2026 — Lecture 2: Requirements with AI

Traian-Florin Șerbănuță

2026

Today's Agenda

1. Frame: architect-critic recap + today's spine
2. Live demo: AI-driven requirements gathering
3. Requirements types: functional / non-functional / domain
4. Prompting for elicitation
5. AI failure modes catalogue
6. Use cases as a structuring lens
7. Bridge to Week 3 + Lab 1 preview

Recap: Architect and Critic

- **Architect / director:** drive AI through the software development lifecycle (SDLC).
- **Critic / reviewer:** read AI's output and name what's wrong or missing.

Last week you saw one cycle of this on a class diagram. Today: same loop, different artifact.

Today's Loop Runs on Requirements

- Architect prompt $\rightarrow$ AI generates a requirements doc.
- Critic reads it $\rightarrow$ finds fabrication, vague non-functional requirements (NFRs), over-spec'ing.
- Architect re-prompts with structure $\rightarrow$ AI tightens.

Each pass costs $\sim$3-5 minutes with current models. In practice you'd iterate 3-5 times before locking the doc.

Four Threads in Today's Lecture

1. Requirements types — functional, non-functional, domain.
2. Prompting for elicitation — how vocabulary shapes AI output.
3. AI failure modes — fabrication, over-specification, vague NFRs.
4. Use cases as a structuring lens.

The demo touches all four; the deeper-dive segments name each one.

The Literacy Floor: Critique, Rationale, Traceability

From Week 1: in the oral defense, *unaided*, you must demonstrate:

- **Critique** — read & critique any AI-generated artifact on the spot.
- **Rationale** — articulate why you directed AI a certain way and what you accepted or rejected.
- **Traceability** — defend the trace across your project: requirement $\rightarrow$ use case $\rightarrow$ class $\rightarrow$ state/sequence $\rightarrow$ test.

Today drills Critique (failure-modes catalogue) and seeds Traceability (traceability starts at requirements).

Demo: Bike-Sharing Requirements

Live AI requirements gathering. Watch for fabrication, vague NFRs, and over-spec'ing.

Prompt to AI: *"Generate a requirements document for a city bike-sharing app: users rent and return bicycles at stations across a city; payment is by app; staff rebalance bikes between stations. Cover functional, non-functional, and domain requirements."*

Three Types of Requirements

Type	Question it answers	One-line example
Functional	What does the system *do*?	"A user can return a bike at any station."
Non-functional	How well does it do it?	"Rental confirmation appears within 2 seconds."
Domain	What external rules apply?	"Bikes must comply with EN 14764 city-bike safety standard."

Functional Requirements: Where AI Fails

- AI fabricates user roles ("city hall liaison", "fleet operations manager") that weren't in your prompt.
- AI conflates concerns: one FR bundles rent + return + report-stolen as a single bullet.
- AI omits the boring core (login, list-bikes, view-history) and reaches for the dramatic edge (theft recovery).
  Watch for: stakeholders that don't trace back to your prompt.

Non-Functional Requirements: Where AI Fails

- "The system shall be performant" — measurable as what?
- "High availability" — 99%? 99.9%? Over what window?
- "User-friendly" — by which standard? Tested how?
  If you can't write a test that proves it, it's not a requirement.

Domain Requirements: Where AI Fails

- AI invents regulations that sound plausible but don't exist ("GDPR Article 47" — there is no Article 47).
- AI overspecifies compliance: 15 separate REQs for one regulation when one umbrella REQ would do.
- AI misses real domain constraints: helmet-law jurisdictions, bike-lane requirements, weather/seasonal closures.
  Watch for: regulations cited without article numbers, or article numbers you can't look up.

Negative Example: Denver Airport Baggage System

Goal: fully automated baggage handling for all airlines.

What went wrong:

- Unclear & changing requirements (scope shifts from airlines).
- Stakeholder misalignment (conflicting airline needs).
- Overly ambitious design (unrealistic automation).
- Poor communication (incomplete, inconsistent docs).

Impact: 16-month delay, $560M cost overrun, system never operational.

Clear, stable, agreed-upon requirements are essential.

Prompting Move #1: Scaffolded > Direct

Direct: *"Give me requirements for a bike-sharing app."*

Returns: a long, unstructured list. Mixes FR/NFR/domain. Fabricates stakeholders.

Scaffolded: *"Give me requirements for a bike-sharing app, organized by use case. Each use case is one user goal; for each, list FR, NFR, and domain constraints separately."*

Returns: structured by intent. Easier to read, critique, and iterate on.

Prompting Move #2: Stakeholder Role Priming

Without priming: *"List non-functional requirements for the bike-sharing station kiosk."*

With priming: *"You are a safety auditor for urban micromobility. List non-functional requirements for the bike-sharing station kiosk from that role."*

The role concentrates AI's output on what that role cares about — safety NFRs become concrete, ones outside the role drop.

> *Roles are filters. Pick the filter that matches what you want AI to surface.*

Prompting Move #3: Negative Prompting

Say what NOT to include:

- *"... no technology choices (no databases, no frameworks)."*
- *"... no fabricated stakeholders — use only roles I named in the prompt."*
- *"... no vague adjectives — every NFR has a measurable threshold."*

This won't catch every failure mode, but it preempts the most common ones.

This Is What Lab 1 Drills

The three moves above — **scaffolded**, **role priming**, **negative prompting** — are exactly what Lab 1 asks you to apply.

Pairs, ~60 min hands-on. **Iterate at least once.** Catch your own failure modes as you go; the 5-line reflection is where you name them.

(Deliverable mechanics: the close slide at the end of the lecture.)

Failure Mode #1: Fabrication

Definition: AI invents requirements (stakeholders, regulations, features) that weren't grounded in your prompt or the domain.

Example: "REQ: integrate with the City Hall API for bike-lane closure notifications" — when no such API was mentioned, and probably doesn't exist.

Critique question: *"Walk this back to my prompt. What did I ask for that produced this?"*

Failure Mode #2: Over-Specification

Definition: AI produces dozens of requirements when the system needs a handful.

Example: 30 separate REQs covering theft prevention, GPS jamming detection, weather sensors, accelerometer alerts, GDPR data-export, audit logs, ... for a bike-sharing station kiosk that mostly rents bikes.

Critique question: *"Read the top 5. Does the system fail without each one? What's load-bearing?"*

Failure Mode #3: Vague Non-Functional Requirements

Definition: NFRs phrased as adjectives with no measurable threshold.

Example: "The system shall be performant." / "User-friendly." / "Highly available."

Critique question: *"Can you write a test that proves this? If not, it's not a requirement."*

Failure Mode #4: Conflated Requirements

Definition: One requirement bundles distinct concerns; success criteria can't apply.

Example: "REQ-12: User can rent, return, and report stolen bikes."

That's three requirements, each with its own success criterion, masquerading as one.

Critique question: *"Split this. What's the test for each piece?"*

Failure Mode #5: Fabricated Technology Choices

Definition: AI puts implementation choices (database, language, framework) into requirements.

Example: "REQ-7: the system shall use Redis for caching."

Redis is not a requirement — it's an implementation. The actual requirement is something like "rental confirmation appears within 2 seconds."

Critique question: *"Why is a database in the requirements? What user need does Redis serve?"*

Your Turn: Name the Failure Mode

Here's one AI-generated requirement:

> *"REQ-7: the system shall use Redis for caching."*

With your neighbour (60s): which of the five failure modes is this, and what's your critique question?

Your Turn: Name the Failure Mode

Here's one AI-generated requirement:

"REQ-7: the system shall use Redis for caching."

With your neighbour (60s): which of the five failure modes is this, and what's your critique question?

Reveal: Fabricated Technology Choices — Redis is an implementation, not a need. Critique question: *"What user need does Redis serve? Where's the measurable requirement underneath?"*

You Saw These in the Demo

The bike-sharing requirements doc cycle 1 had (pick what actually appeared):

- Fabricated stakeholders (likely)
- Vague NFRs (very likely)
- Over-specification (very likely)
- Fabricated technology (possible)

Cycle 2 — with the use-case scaffold — dropped most of them. **Scaffolds are how the architect-half corrects the critic-half's findings.**

Use Case = One User Goal

A **use case** is one thing a user wants to accomplish with the system.

For the bike-sharing app:

- *Rent a bike* (one user goal)
- *Return a bike* (another user goal)
- *Report a stolen bike* (another user goal)

Each use case is a hook — you can list FR, NFR, and domain requirements *per use case*, instead of in one giant flat list.

Use Cases as a Scaffold for AI

Look at what happened in the demo:

Prompt #1 (flat): "Generate requirements for a bike-sharing app." → Long unstructured list, fabricated stakeholders, vague NFRs.

Prompt #2 (use-case-scaffolded): "Rewrite organized by use case. For each: who, success criterion, one NFR." → Tighter output. Each requirement now traces to one user goal.

> *The scaffold gives AI a vocabulary; vocabulary tightens output.*

UML Notation Is Coming — In Week 6

Today: "use case" as a structural noun. No diagrams.

In **Week 6** (Behavioral I): the UML notation — actors, ovals, system boundary, include / extend / generalization relationships. Plus sequence diagrams.

Until then: think of use cases as labeled buckets for requirements, nothing more.

This Week's Lab: Lab 1

- Week 2 lab slot: tooling onboarding (~15 min) + AI-driven requirements drill in pairs (~60 min) + share-out (~25 min).
- Domain assigned at lab start (toy domain, not your project).
- Deliverable: requirements doc + 5-line reflection on the failure modes you observed.
- Commit to the course lab repo (instructions handed out at lab start).

Bring a laptop with Continue.dev pre-installed per `tooling/SETUP.md`.

Next Week: Testable Specs

If AI cannot produce a passing test from your spec, your spec is too vague.

Week 3 reframes the requirements loop as a test-generation loop. The mantra above is the heuristic; Week 3 turns it into a discipline.

Critique, Rationale, Traceability — The Through-Line

Today you drilled:

- **Critique** — read AI's requirements output and named the failure modes.
- **Traceability** — saw why traceability starts at requirements: if requirements are fabricated, every downstream artifact inherits the fabrication.

Rationale (articulating *why* you directed AI a certain way) lands as you produce your own logs in Lab 1.

That's It For Today

- Next lecture (Week 3): testable specs and test-driven development (TDD)-as-spec.
- This week's lab (Lab 1): drive AI to produce a requirements doc, log what failed.

Questions?